Settle
Technical design

Project repository: https://github.com/Vijay-1807/settle-reconciliation
The repository is public and includes the commit history.

Portfolio: vijaybontha.vercel.app
LinkedIn: linkedin.com/in/bonthavijay
GitHub: github.com/Vijay-1807

How to try it: The full app runs locally at http://localhost:3000. Setup steps and demo logins are in deployment.md. There is no live hosted version; hosting is optional in the brief.

1. What the app does

Settle helps the finance team check whether each order was paid correctly. Staff upload reports, check the suggested column mapping and approve the import. The app calculates expected payments, flags differences and puts the supporting details in a claims list.

The core screens cover uploads, saved mappings, order matching, rate cards, claims and money at risk. The build also includes courier weight slabs, warehouse receipts, separate TCS/TDS totals, partial payouts and a review queue for missing order IDs.

2. How it is built

Part	Choice and reason
Frontend	JavaScript and custom CSS, with DM Sans and Manrope served locally. Lucide supplies the icons; SheetJS reads CSV and Excel files. A small frontend keeps this focused app easy to follow.
Backend	A shared HTTP API checks sign-in, roles and inputs before changing data. Financial rules are kept in engine.mjs so they can be tested without the browser.
Database	SQLite stores orders, settings and sessions locally. An optional Worker/D1 build uses the same API. esbuild prepares the browser and Worker files.

From a report to an order record

3. The data model

Record	What it stores
Orders	Each order ID is unique. Its record holds channel, value, payment type, dates, partner, receipts, freight, tax deductions, claim status and source evidence. A payout ledger tracks each report's contribution.
Workspace settings	Channel and courier cards, mapping versions, upload history, match reviews and activity history. A mapping records its source, report type, columns, version, approver and date.
Sessions	A random session ID, demo user details, role and expiry. The session ID is stored in an HTTP-only cookie.

Order data is stored as JSON inside SQLite records. This keeps the take-home build small. Imports and mapping history are saved in one transaction: if a database write fails, all changes in that import are rolled back. One server handles writes one at a time. Several running servers would need stronger checks against conflicting updates.

4. Upload and approval

1. Choose the report type and source. Upload a CSV or the first sheet of an Excel file, up to 25 MB and 100,000 rows.
2. Load a saved mapping or get suggestions from the header names.
3. Review the mapped fields and first 20 rows. Changing a mapping clears the approval checkbox.
4. Check every row before saving. Invalid amounts, dates or duplicate snapshot rows return row-level errors. Missing order IDs go to manual review.
5. Save the valid order changes, review items, mapping version and history together. Recalculate the dashboard from the saved records.

5. Partial payments and evidence

Per-order totals mode replaces the previous received amount. Payout transactions mode adds partial payments and groups repeated order rows. A contribution is identified by source, report type and filename. Uploading the same report again corrects that contribution rather than counting it twice. Reviewed rows also use their source row number.

Each import keeps the mapped financial fields and source rows as evidence. Claims can be marked Open, In progress, Recovered or Rejected. The export includes the calculation, deductions and report details. A warehouse receipt clears the missing-return flag. The app does not move money or send claims to partners.

How an order gets checked

6. Financial rules

For a delivered order, expected payment is order value minus commission, fixed fee, TCS and TDS. Reported tax deductions are used when supplied; otherwise the rate card gives an estimate. Tax components are rounded to two decimals before subtraction. TCS and TDS are shown separately and are not treated as leakage.

Returned orders use the agreed return fee as a negative expected amount. Courier cards calculate freight from actual package weight, rounding extra weight up to the next slab. A gap must be more than ₹10 to be flagged. A zero payment becomes overdue after the settlement window. A missing return is flagged after its receipt window. Positive payment differences stay visible but do not create a shortage claim.

Brief's example	Expected	Received	Difference
ORD-W-204118	₹1,409	₹0	-₹1,409
ORD-M-88120	₹899	₹862	-₹37
ORD-W-204377	-₹96	-₹142	-₹46

7. Mapping suggestions and mistakes

The mapping helper is a small local nearest-neighbour model, not an LLM. It compares short groups of letters in column headers against known examples. Approved mappings add more examples. Strong matches get a column before weaker matches, and unclear suggestions are left for the user to choose. Scores measure similarity; they are not probabilities.

Every mapping needs a preview and approval before saving. Versions can be restored by the co-founder. If an order ID is missing, the app suggests up to five possible orders using shipment reference, value, channel, date and payment amount. A person must choose or enter the order and confirm it. Suggestions never update an order or close a claim by themselves.

8. Basic security

The API checks sessions and roles on the server. Prepared SQL protects database queries; input checks reject unexpected fields and invalid values. Writes check their request origin. UI text is escaped and CSV exports guard against spreadsheet formulas. Only approved financial fields are saved, so customer names, phones and addresses are not needed. Shared demo accounts use made-up data. Proper account management and recovery controls are still needed for a real rollout.

Built for the finance team's work

9. UI choices

Screen	Why it is laid out this way
Overview	Shows money at risk, payments received, completed checks and recoveries. Partner bars and next actions help staff choose what to follow up. The figures come from order records.
Reconciliation	Simple search and filters sit above a paginated table. An order drawer explains each calculation. The export follows the selected tab. Missing-ID matches have their own review tab.
Imports and mapping history	The upload flow separates file choice, mapping review and approval. Users can see 20 rows before saving, inspect old versions and restore a version when needed.
Claims and returns	Selected rows are clearly marked. Changing a filter clears the selection to avoid updating hidden orders. Evidence export and warehouse receipt entry sit close to the records.
Rate cards	Only the co-founder can change rates. Edited cards show Unsaved and enable the save button. Courier cards explain the first and additional weight slabs.

The design uses white space, dark text and one restrained green accent. Numbers line up in tables. Exception labels use words as well as colour. Phone layouts use a menu, stacked panels and scrollable tables. Larger touch targets, visible keyboard focus, dialog focus handling and reduced-motion support make the app easier to use.

10. What was checked

19 logic and API tests passed. These cover the supplied examples, tax separation, due dates, courier slabs, mapping mistakes, roles, invalid inputs, rollback, partial payments and manual match approval.

10 Playwright tests passed across desktop and phone layouts. They check navigation, fonts, upload approval, matching, downloads, review decisions, role switching and selection after filtering. Tests use a separate database. GitHub Actions also passed on Linux.

The recorded local 100,000-row run took 5.3 seconds. It read the CSV, checked rows, saved them in SQLite, matched orders and calculated totals. This does not include network transfer or browser rendering. The rules-only check took 209 ms. Hosted performance has not been measured.

What is assumed and what comes next

11. Assumptions

A1. Sample data. The 480 orders are made up. The reference date is fixed at 2 November 2026 so the same review gives the same results. Six channels and two placeholder couriers represent the brief.
A2. Dates and rates. Settlement and return windows start from order date as a proxy and default to seven days. Rates are examples. Marketplace 1 uses a ₹292.80 fixed fee, no commission, 0.5% TCS and 0.1% TDS to reproduce the brief's ₹899 result. These are sample inputs, not tax advice.
A3. Payout reports. Corrected transaction files keep the same source, type and filename. Renamed or overlapping reports must not be added as separate payments.
A4. Freight and returns. Missing actual weight defaults to 0.5 kg. Channel freight is a fallback when no courier card exists. Missing-return risk uses order value. Freight invoices are assumed separate from settlement deductions.
A5. Files. Excel uses its first sheet and requires ISO dates. Original files stay on the device; mapped fields and source rows are saved. More than 500 unmatched rows rejects the import so the source file or mapping can be checked.
A6. Demo scope. Shared accounts and local performance results apply to this demo. The 98% matching, three-day close and ₹30 lakh recovery figures are business goals; they have not been proved with real customer data.

12. Limits and next steps

Rate changes currently recalculate old orders. A real accounting rollout needs dated agreements and an unchangeable history. The next work would add courier zones, background imports, database filtering for large tables, checks for updates from several servers, proper account management, backup recovery and hosted load tests. A compressed Excel file can expand substantially, so production parsing also needs stronger resource limits.

Storefront and inventory sync, GST filing, Tally integration, payment execution and automatic claim filing are not included. The core brief is implemented; these items are outside this take-home slice.

13. Review and submission

The repository is public and keeps the commit history. The submission ZIP contains exactly this PDF and deployment.md. The setup guide includes commands, demo logins, environment names and a walkthrough. The reviewer can run the complete app locally. Keep the repository public until review finishes. The assessment upload and final Submit click are separate from this build.